Sistema de Facturación e Inventario Multisede — *El Bodegón de la Tecnología*

Aplicación web progresiva (PWA) para la gestión de facturación, inventario, caja y compras de un negocio minorista de tecnología con varias sedes.

Tabla de contenido

1. Introducción
2. Objetivo general
3. Objetivos específicos
4. Marco teórico
5. Diagnóstico
6. Propuesta
7. Implementación
8. Justificación del proyecto

---

1. Introducción

1.1 Contexto

El comercio minorista de tecnología y repuestos electrónicos en ciudades intermedias de Colombia opera bajo un modelo particular: negocios familiares o de pequeña sociedad, con una o varias sedes en municipios cercanos, un catálogo de cientos o miles de referencias de bajo valor unitario y alta rotación, y un equipo reducido de vendedores que atienden directamente al público. La operación diaria combina ventas de contado, ventas fiadas a clientes conocidos, encargos de repuestos que no se tienen en existencia, traslados de mercancía entre sedes cuando a una se le agota lo que a la otra le sobra, y un cierre de caja al final de cada turno para verificar que el efectivo contado corresponde con lo vendido.

El Bodegón de la Tecnología es un negocio de este tipo. Cuenta con dos puntos de venta —**Ocaña** y **Aguachica**—, un administrador que supervisa ambos y vendedores que operan cada sede. Esta estructura, aparentemente simple, genera un problema de información que crece de manera no lineal: cada sede mantiene su propio inventario físico, su propia numeración de facturas, su propia caja y sus propios clientes con deudas pendientes, pero el negocio es uno solo y las decisiones de compra, precios y reposición se toman de forma centralizada. Sin un sistema que consolide esa información, el

administrador debe reconstruir manualmente el estado del negocio a partir de fuentes dispersas: cuadernos, mensajes de WhatsApp, hojas de cálculo y llamadas telefónicas.

1.2 El problema que resuelve

El problema central que aborda este proyecto es **la ausencia de una fuente única y confiable de información operativa en un negocio minorista con múltiples sedes**, y las consecuencias concretas que esa ausencia produce en la operación diaria. Este problema general se manifiesta en cinco síntomas verificables:

a) Inventario desincronizado entre sedes. Cuando el registro de existencias no se actualiza en el momento mismo de la venta, el stock que figura en los apuntes deja de corresponder con el stock físico en cuestión de días. El vendedor pierde la capacidad de responder con certeza si un producto está disponible, y el administrador pierde la capacidad de saber qué debe comprar. En un catálogo de alta rotación, esta desviación se acumula: cada venta no registrada, cada traslado informal entre sedes y cada devolución sin anotar amplían la brecha entre lo que el sistema dice y lo que hay en la estantería.

b) Cierre de caja sin trazabilidad. El cuadre de caja al final del turno consiste en comparar el efectivo contado contra el efectivo que *debería* haber. Sin un registro que vincule cada venta con el turno en que se hizo, con su método de pago y con los descuentos aplicados, ese cálculo se vuelve una estimación. Cuando aparece un faltante, no existe manera de reconstruir su origen: pudo ser un error de digitación, un descuento no autorizado, una venta cobrada por transferencia y anotada como efectivo, o una diferencia real. La imposibilidad de auditar el turno convierte cada descuadre en un conflicto de confianza entre el administrador y el vendedor.

c) Cartera de créditos sin control. La venta fiada es una práctica comercial necesaria en este segmento: sostiene la relación con el cliente recurrente y es, en la práctica, una condición para competir. Sin embargo, cuando el saldo pendiente de cada cliente vive en un cuaderno, la cartera se vuelve invisible como conjunto. No se sabe cuánto dinero total está prestado, quién debe hace cuánto, ni qué abonos se han recibido. El capital de trabajo del negocio queda inmovilizado en una cifra que nadie conoce con exactitud.

d) Decisiones de compra por intuición. Determinar qué reponer exige cruzar dos datos que normalmente no están disponibles al mismo tiempo: cuánto queda de cada producto y con qué velocidad se está vendiendo. Sin ese cruce, la decisión de compra se toma por memoria o por el reclamo del vendedor que se quedó sin existencias. El resultado son las dos caras del mismo error: capital inmovilizado en referencias que no rotan, y ventas perdidas por agotados en las referencias que sí rotan.

e) Autorizaciones informales. Cambiar el precio de un producto, trasladar mercancía a la otra sede o aplicar un descuento son decisiones que corresponden al administrador, pero que se solicitan y se conceden por mensaje o por teléfono, sin quedar registradas. Esto produce dos efectos: el vendedor

queda bloqueado mientras espera respuesta, y la decisión —una vez tomada— no deja rastro que permita revisarla después.

1.3 Para quién lo resuelve

El sistema atiende a tres perfiles de usuario, cada uno con necesidades y permisos distintos, modelados en la aplicación mediante los roles ADMIN, CASHIER y SUPERVISOR:

Rol	Quién es	Qué necesita resolver
Administrador (ADMIN)	El dueño o gerente que responde por ambas sedes.	Ver el estado consolidado del negocio, conocer la ganancia real, decidir qué comprar, autorizar cambios de precio y traslados, y auditar los cierres de caja.
Vendedor / Cajero (CASHIER)	Quien atiende el mostrador en una sede.	Facturar rápido, consultar existencias reales, registrar créditos y abonos, gestionar encargos, y cerrar su turno con un cuadre que pueda respaldar.
Supervisor (SUPERVISOR)	Perfil de consulta sobre ambas sedes, sin capacidad de modificación.	Revisar la operación y los informes sin riesgo de alterar datos.

De forma indirecta, el **cliente final** también resulta beneficiado: obtiene respuestas certeras sobre disponibilidad, un comprobante de compra formal, un registro verificable de sus abonos cuando compra a crédito, y seguimiento del estado de los repuestos que encarga.

1.4 Alcance del documento

Este documento presenta el proyecto de forma integral: el diagnóstico de la situación que lo origina, los objetivos que persigue, el marco teórico que sustenta las decisiones técnicas —arquitectura, lenguajes, motor de base de datos y patrones aplicados, con la justificación de cada elección—, la propuesta de solución, la implementación realizada y la justificación de su desarrollo.

---

2. Objetivo general

El proyecto persigue dos objetivos generales:

1. **Centralizar en una única plataforma web la operación comercial de un negocio minorista con múltiples sedes**, de modo que la facturación, el inventario, la caja, la cartera de créditos y los encargos de clientes se registren en una sola fuente de datos consistente, accesible en tiempo real desde cualquier sede y con la información segregada según el rol de cada usuario.

1. **Convertir el registro operativo en información para la toma de decisiones**, entregando al

administrador indicadores de ventas, márgenes de ganancia, rotación de productos y estado del inventario que le permitan decidir qué comprar, qué precios ajustar y cómo distribuir la mercancía entre sedes con base en datos y no en estimaciones.

---

3. Objetivos específicos

1. **Implementar un módulo de facturación** que registre ventas de contado y a crédito, admita múltiples métodos de pago (efectivo, Nequi, Bancolombia) en una misma factura, aplique descuentos con motivo justificado y descuente el inventario de forma atómica en el momento de la venta.

1. **Desarrollar la gestión de inventario por sede**, con existencias independientes por punto de venta, registro trazable de cada movimiento (venta, devolución, cambio, traslado, entrada y salida de stock) y carga masiva desde archivos de Excel para las operaciones de alto volumen.

1. **Construir el módulo de caja por turnos**, con apertura sobre una base en efectivo, vinculación automática de cada venta al turno en que se realizó, y cierre con un informe que contraste el efectivo esperado contra el efectivo contado y explique la diferencia.

1. **Gestionar la cartera de créditos**, generando automáticamente un préstamo por cada venta fiada, registrando los abonos parciales con su método de pago y manteniendo el saldo pendiente por cliente actualizado.

1. **Implementar los flujos de autorización** para traslados de mercancía entre sedes y para cambios de precio, bajo un esquema de solicitud pendiente → aprobación o rechazo por parte del administrador, con registro de quién resolvió y cuándo.

1. **Desarrollar un motor de recomendación de compras** que analice el stock actual y la velocidad de venta de los últimos 30 días para proyectar agotamientos y sugerir qué productos reponer con prioridad.

1. **Gestionar pedidos especiales** de repuestos no inventariados, con seguimiento por estados (abonado → pedido → llegó → entregado), control de abonos y saldos, y costo privado visible solo para el administrador.

1. **Generar informes gerenciales** de ventas por día, productos más vendidos, estado del inventario, distribución por método de pago, ganancia por venta y comparativo entre sedes.

1. **Garantizar la seguridad del acceso** mediante autenticación con JSON Web Tokens de vida corta, rotación de tokens de refresco almacenados en base de datos y control de autorización por rol en cada endpoint.

1. **Entregar la aplicación como PWA instalable**, con caché de recursos e interfaz responsiva, de modo que funcione como aplicación nativa en el dispositivo del vendedor sin necesidad de

distribución por tiendas.

1. **Integrar notificaciones automáticas por Telegram** que informen al administrador la apertura y el cierre de caja de cada sede con el informe completo del turno.

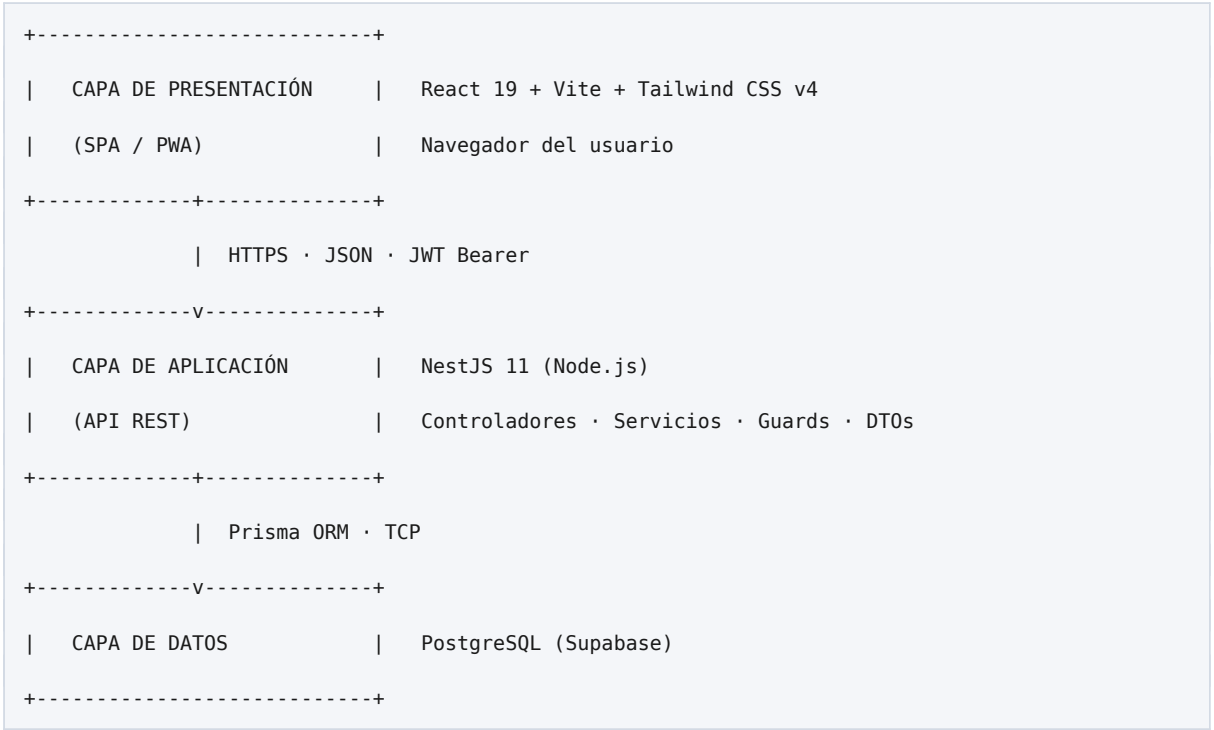
---

4. Marco teórico

Esta sección expone los fundamentos conceptuales y tecnológicos sobre los que se construye el sistema, junto con la justificación de cada decisión.

4.1 Arquitectura general: cliente-servidor desacoplado

El sistema adopta una **arquitectura cliente-servidor de tres capas con separación estricta entre frontend y backend**, comunicadas mediante una API REST sobre HTTP/JSON.



¿Por qué desacoplado y no monolítico con vistas del lado del servidor? Un monolito con renderizado en servidor (por ejemplo, Laravel con Blade o Django con plantillas) habría sido más rápido de arrancar, pero impide dos requisitos del proyecto: que la aplicación se instale como PWA en el dispositivo del vendedor y funcione con caché ante conexión intermitente, y que la misma API pueda servir en el futuro a otros clientes (una aplicación móvil nativa, un lector de códigos de barras dedicado o una integración con facturación electrónica DIAN). La separación tiene un costo —dos proyectos, dos despliegues, un contrato de API que mantener— que se asume conscientemente a cambio de esa flexibilidad.

¿Por qué REST y no GraphQL? El dominio tiene entidades bien delimitadas y consultas

predecibles. GraphQL aporta valor cuando el cliente necesita componer consultas variables sobre grafos complejos; aquí introduciría complejidad (resolvers, problema N+1, caché) sin beneficio proporcional.

4.2 Backend: NestJS sobre Node.js

NestJS 11 es el framework del servidor. Es un framework de Node.js construido sobre Express que impone una arquitectura modular inspirada en Angular, con inyección de dependencias, decoradores y separación en controladores, servicios y módulos.

Justificación de la elección:

- **Arquitectura impuesta por convención.** Node.js con Express puro no prescribe estructura; en proyectos que crecen, esa libertad degenera en organización inconsistente. NestJS obliga a la tríada controlador → servicio → módulo, donde el controlador solo traduce HTTP, el servicio contiene la lógica de negocio y el módulo declara las dependencias. En este proyecto son 16 módulos independientes que se registran en un único AppModule; agregar una funcionalidad nueva no exige tocar las existentes.
- **Inyección de dependencias nativa.** El contenedor de IoC resuelve las dependencias entre servicios de forma declarativa, lo que permite sustituirlas por dobles de prueba sin modificar el código productivo.
- **TypeScript de primera clase.** El framework está escrito en TypeScript y aprovecha decoradores y metadatos, lo que habilita validación declarativa y guards por metadatos.
- **Guards e interceptores como transversales.** La autenticación y la autorización se aplican como Guards reutilizables (JwtAuthGuard, RolesGuard) declarados sobre controladores completos, en lugar de repetir verificaciones dentro de cada método.
- **Un solo lenguaje en toda la pila.** Cliente y servidor en TypeScript eliminan el cambio de contexto y permiten compartir la definición de tipos del dominio.

4.3 Lenguaje: TypeScript

Todo el código —frontend y backend— está escrito en **TypeScript** con configuración estricta (strictNullChecks, noUnusedLocals, noUnusedParameters, verbatimModuleSyntax).

Justificación: en un sistema que maneja dinero, inventario y cartera, un error de tipos no es un detalle estético sino un riesgo financiero. El tipado estático detecta en compilación errores que JavaScript solo revelaría en ejecución: un undefined en un precio, un identificador de producto tratado como número cuando es un UUID, un campo faltante en el cuerpo de una petición. El compilador actúa como una primera capa de pruebas permanente y sin costo de mantenimiento.

4.4 Persistencia: PostgreSQL y Prisma ORM

PostgreSQL es el motor de base de datos, alojado en **Supabase**.

Justificación de PostgreSQL frente a alternativas:

- **Transacciones ACID.** Registrar una venta implica varias escrituras que deben ocurrir todas o ninguna: crear la factura, sus líneas de detalle, sus pagos, descontar el inventario, generar los movimientos de stock y —si es fiada— crear el préstamo. Una escritura parcial dejaría el inventario corrompido. PostgreSQL garantiza atomicidad mediante transacciones, propiedad que una base de datos documental como MongoDB ofrece con más restricciones y menos naturalidad.
- **Integridad referencial declarativa.** Las claves foráneas y las restricciones de unicidad se hacen cumplir en el motor, no en la aplicación. Restricciones como @@unique([branchId, invoiceNumber]) —que garantiza numeración de facturas consecutiva y sin repetir por sede— o @@unique([branchId, productId]) —que impide dos registros de existencias del mismo producto en la misma sede— son invariantes del dominio que el motor protege aunque la aplicación tenga defectos.
- **Tipo Decimal para valores monetarios.** Los importes se almacenan como Decimal y no como punto flotante, evitando los errores de redondeo acumulativo que produce float en operaciones financieras.
- **Modelo relacional acorde al dominio.** El dominio es inherentemente relacional: una empresa tiene sedes, una sede tiene usuarios, clientes e inventario; una venta tiene detalles, pagos y eventualmente un préstamo. Forzar esta estructura en un modelo documental exigiría duplicación o referencias manuales.

Prisma 7 es el ORM. Define el esquema en un archivo declarativo (schema.prisma) desde el cual genera un cliente TypeScript completamente tipado y las migraciones de base de datos.

Justificación frente a alternativas: comparado con TypeORM, Prisma ofrece un cliente generado cuyos tipos derivan del esquema real —si se renombra una columna, el compilador señala todos los puntos afectados—; comparado con SQL directo, elimina el riesgo de inyección y el trabajo repetitivo de mapeo, sin impedir consultas SQL crudas donde el rendimiento lo exige (como en el cálculo agregado de ganancias, resuelto en SQL para no cargar miles de registros en memoria). El sistema de migraciones versiona la evolución del esquema y la hace reproducible.

4.5 Frontend: React 19 + Vite + Tailwind CSS

React 19 como biblioteca de interfaz, **Vite 8** como herramienta de construcción y **Tailwind CSS v4** para los estilos.

Justificación:

- **React** y su modelo de componentes permiten construir una interfaz por composición. El proyecto explota esto con una biblioteca interna de componentes reutilizables (Card, Modal, StatCard, Panel, TabPills, ProductSelect, StatusBadge) que garantiza consistencia visual y evita duplicación entre las diez páginas de la aplicación.
- **Vite** ofrece arranque casi instantáneo y recarga en caliente mediante módulos ES nativos en desarrollo, y compilación optimizada con división de código en producción. Frente a

Webpack, reduce drásticamente el tiempo de espera del desarrollador.

- **Tailwind CSS v4** aplica la metodología *utility-first*: los estilos se declaran como clases atómicas en el marcado. Esto elimina la deriva de hojas de estilo globales —donde nadie sabe si una regla sigue en uso— y produce un CSS final que contiene únicamente las utilidades realmente empleadas.
- **Zustand** para el estado global (sesión del usuario y sede seleccionada). Frente a Redux, evita el ceremonial de acciones, reductores y middlewares para un estado que es deliberadamente pequeño; frente al Context API de React, evita re-renderizados innecesarios porque los componentes se suscriben a porciones específicas del estado.

4.6 Aplicación web progresiva (PWA)

La aplicación se distribuye como **PWA instalable** mediante vite-plugin-pwa, que genera un *service worker* basado en Workbox con registro autoUpdate, manifiesto de aplicación e iconos adaptables.

Justificación: el vendedor trabaja desde un teléfono o una tableta en el mostrador. Una PWA se instala en la pantalla de inicio y se ejecuta en modo standalone —sin barra de navegador—, ofreciendo la experiencia de una aplicación nativa sin el costo de desarrollar y mantener versiones separadas para Android e iOS, ni la fricción de publicar en tiendas de aplicaciones. La estrategia de caché NetworkFirst sobre las llamadas a la API prioriza el dato fresco pero conserva la última respuesta válida, lo que mantiene la aplicación utilizable ante conexión intermitente.

4.7 Seguridad: autenticación JWT con rotación de tokens

El control de acceso se basa en **JSON Web Tokens** con un esquema de doble token:

- **Token de acceso**: vigencia de 15 minutos, firmado con JWT_SECRET, transportado en la cabecera Authorization: Bearer. Su carga útil contiene el identificador del usuario y su rol.
- **Token de refresco**: vigencia de 7 días, almacenado como UUID aleatorio en la tabla RefreshToken y **rotado en cada uso** (el token anterior se elimina y se emite uno nuevo).
- **Contraseñas** cifradas con bcrypt.

Justificación del diseño: un JWT puro no puede revocarse antes de su expiración, lo que obliga a elegir entre tokens de larga vida (inseguros) o sesiones incómodas. El esquema de doble token resuelve el dilema: el token de acceso es tan corto que su robo tiene ventana mínima, mientras que el token de refresco vive en base de datos y por tanto **sí puede revocarse** eliminando el registro. La rotación agrega detección de reutilización: si un token ya consumido vuelve a presentarse, es señal de compromiso.

La **autorización por rol** se aplica con RolesGuard y el decorador @Roles() sobre cada endpoint, y se refuerza en el cliente restringiendo las rutas accesibles según el rol —el cajero solo alcanza caja, facturación y sus módulos de solicitudes—. La validación de las peticiones se realiza con class-validator mediante un ValidationPipe global configurado con whitelist: true, que descarta cualquier campo no declarado en el DTO correspondiente.

4.8 Patrones de diseño aplicados

Patrón	Dónde se aplica	Para qué
Inyección de dependencias	Servicios NestJS	Desacoplar componentes y habilitar pruebas
Repositorio (vía Prisma)	Acceso a datos	Aislar la lógica de negocio del SQL
DTO + validación declarativa	Entrada de cada endpoint	Validar y sanear en la frontera del sistema
Guard / Middleware	Autenticación y roles	Aplicar seguridad transversalmente
Módulo global	PrismaModule con @Global()	Inyectar la conexión sin reimportar en cada módulo
Máquina de estados	Solicitudes y pedidos especiales	Modelar flujos de aprobación y seguimiento
Transacción de unidad de trabajo	Ventas, traslados, ajustes masivos	Garantizar consistencia en operaciones compuestas
Componentes de presentación	Biblioteca components/ui/	Reutilización y consistencia visual

4.9 Modelo de dominio

El esquema define **19 modelos** y **10 enumeraciones** que representan el negocio:

```
Company (Empresa)
  +-- Branch (Sede)
        +-- User (ADMIN | SUPERVISOR | CASHIER)
        +-- Customer (REGULAR | SPECIAL)
        +-- Inventory (existencias Producto × Sede)
        |     +-- InventoryMovement (SALE | RETURN | EXCHANGE |
        |                            BRANCH_TRANSFER | STOCK_IN | STOCK_OUT)
        +-- CashSession (turno de caja: OPEN | CLOSED)
        +-- SpecialOrder (DEPOSITED -> ORDERED -> ARRIVED -> PICKED_UP)
        |     +-- SpecialOrderPayment (DEPOSIT | FINAL)
        +-- PurchaseListItem (MANUAL | AUTO)
```

```
Product ---> Category

Product ---> PriceChangeRequest (PENDING -> APPROVED | REJECTED)

Product ---> BranchTransferRequest (PENDING -> APPROVED | REJECTED)


Sale (por sede, por usuario, vinculada a un turno de caja)

  +-- SaleDetail (producto × cantidad × precio × descuento)

  +-- SalePayment (CASH | NEQUI | BANCOLOMBIA — varios por venta)

  +-- Loan (si isCredit = true)

        +-- LoanPayment


User ---> TelegramLink (chats vinculados)

User ---> RefreshToken
```

Decisiones de modelado relevantes:

- **Congelación de precios.** SaleDetail.unitPrice almacena el precio efectivamente cobrado, no una referencia al producto. Un cambio posterior de precio no altera el histórico de ventas ni distorsiona los informes de ganancia.
- **Precio al detal y al por mayor.** Cada producto mantiene ambos precios; en el momento de facturar se elige uno por línea y el valor queda congelado en el detalle.
- **Trazabilidad de inventario por eventos.** Cada modificación de existencias genera un InventoryMovement tipado. El stock actual es verificable contra la suma de sus movimientos, lo que permite auditoría sin sistemas externos.
- **Vinculación de traslados.** Los dos movimientos de una transferencia (salida del origen, entrada al destino) comparten un transferGroupId que los identifica como una sola operación.
- **Vinculación venta–turno.** Sale.cashSessionId asocia cada venta al turno de caja en que ocurrió, requisito indispensable para que el cuadre sea auditable.

---

5. Diagnóstico

5.1 Método

El diagnóstico se construyó a partir de la observación de la operación diaria en ambas sedes, entrevistas con el administrador y los vendedores, y revisión de los soportes existentes: cuadernos de fiado, apuntes de cierre de caja y hojas de cálculo de inventario.

5.2 Situación encontrada

Área	Situación previa	Consecuencia medible
Facturación	Talonario físico y anotaciones sueltas. Numeración no consecutiva.	Sin histórico consultable. Imposible reconstruir qué se vendió, a quién y en qué condiciones.
Inventario	Hoja de cálculo desactualizada, sin división por sede.	El vendedor no puede confirmar disponibilidad; se prometen productos agotados y se pierden ventas de productos que sí había.
Caja	Conteo manual al cierre, sin respaldo de las ventas del turno.	Los descuadres no se pueden explicar; se atribuyen a error humano sin evidencia y generan fricción con el personal.
Créditos	Cuaderno de fiados por sede.	Cartera total desconocida. Abonos que se olvidan de registrar. Deudas que prescriben de hecho por falta de seguimiento.
Traslados entre sedes	Acuerdo verbal por teléfono o WhatsApp.	Mercancía que sale de una sede sin registrarse como entrada en la otra; el inventario de ambas queda desviado.
Cambios de precio	Autorización verbal del administrador.	Sin registro de quién autorizó qué, ni de cuándo. Precios distintos para el mismo producto según el vendedor.
Pedidos especiales	Anotación en papel con el abono recibido.	Encargos que se pierden, abonos que se reclaman sin soporte, clientes que preguntan por un pedido cuyo estado nadie conoce.
Compras	Decisión por memoria o por reclamo del vendedor.	Sobrestock de referencias lentas y agotados recurrentes de referencias rápidas, simultáneamente.
Informes	Inexistentes.	No se conoce la ganancia real, ni qué productos la generan, ni qué sede rinde mejor.

5.3 Problemas críticos identificados

1. **Ausencia de fuente única de verdad.** La información del negocio está fragmentada en soportes que nadie consolida.
2. **Imposibilidad de auditoría.** Ninguna operación deja rastro suficiente para ser reconstruida después.

3. **Ceguera para la decisión.** Las decisiones de compra, precio y distribución se toman sin datos.
4. **Dependencia de la memoria individual.** El conocimiento operativo reside en las personas; su ausencia paraliza la operación.
5. **Fricción entre sedes.** La falta de un canal formal convierte cada coordinación en una negociación informal.

---

6. Propuesta

6.1 Solución planteada

Se propone el desarrollo de una **plataforma web progresiva de gestión comercial multisede**, con base de datos centralizada en la nube, acceso diferenciado por rol y disponible desde cualquier dispositivo con navegador, que sustituya integralmente los soportes manuales descritos en el diagnóstico.

6.2 Módulos propuestos

1. Autenticación y control de acceso. Inicio de sesión con usuario y contraseña, tokens de vida corta con rotación, y tres roles con permisos diferenciados. El cajero queda restringido a su sede y a los módulos de su competencia.

2. Facturación. Registro de ventas con búsqueda de producto por nombre o lectura de código de barras, selección de precio al detal o al por mayor por línea, descuentos con motivo obligatorio, pago combinado en varios métodos, y venta a crédito con generación automática del préstamo. La venta descuenta inventario y genera sus movimientos dentro de una única transacción.

3. Inventario por sede. Existencias independientes por punto de venta, con entradas, salidas y traslados; registro trazable de cada movimiento con posibilidad de corrección; alerta de stock bajo; y carga masiva desde Excel para operaciones de alto volumen.

4. Caja por turnos. Apertura con base en efectivo declarada, vinculación automática de las ventas del turno, y cierre con informe que contrasta el efectivo esperado —base más efectivo recibido más abonos de pedidos especiales— contra el efectivo contado, con desglose por método de pago, préstamos otorgados y descuentos aplicados.

5. Clientes y cartera. Registro de clientes por sede, préstamo automático por venta fiada, abonos parciales con método de pago, saldo pendiente actualizado, y vista de clientes con deuda agrupada.

6. Traslados entre sedes. Solicitud del vendedor que no altera el inventario, resolución del administrador (aprobación o rechazo), y aplicación del movimiento de transferencia solo tras la aprobación.

7. Solicitudes de cambio de precio. Propuesta del vendedor con precio sugerido y motivo; el precio

del producto no cambia hasta que el administrador aprueba, quedando registrado quién resolvió y cuándo.

8. Pedidos especiales. Registro de encargos no inventariados, con valor total, abono, saldo, fecha estimada de llegada y avance por estados. El costo del repuesto es visible únicamente para el administrador y alimenta el cálculo de ganancia.

9. Lista de compras con recomendación automática. Motor que analiza existencias y velocidad de venta de los últimos 30 días para detectar productos agotados, en stock bajo, de alta rotación o con agotamiento proyectado dentro de siete días. Las sugerencias se ordenan por urgencia y pueden confirmarse para incorporarse a la lista, que se exporta en PDF orientado al proveedor.

10. Informes gerenciales. Ventas por día, productos más vendidos, estado del inventario, distribución por método de pago, ganancia por venta —calculada como el cobrado menos el costo de compra de cada línea— y comparativo entre sedes.

11. Notificaciones por Telegram. Vinculación de la cuenta con uno o varios chats mediante código de un solo uso, y envío automático al administrador de la apertura y el cierre de caja con el informe completo del turno.

6.3 Beneficios esperados

Problema del diagnóstico	Cómo lo resuelve la propuesta
Inventario desincronizado	El descuento es automático y transaccional en cada venta; todo movimiento queda tipado y trazado.
Cierre de caja sin trazabilidad	Cada venta se vincula a su turno; el informe de cierre desglosa y explica la diferencia.
Cartera sin control	El préstamo se genera solo, los abonos quedan registrados y el saldo se actualiza.
Decisiones por intuición	El motor de recomendación cruza stock y rotación; los informes muestran márgenes reales.
Autorizaciones informales	Los flujos de aprobación dejan registro de solicitante, resolutor y fecha.
Coordinación entre sedes	El traslado es una solicitud formal que solo mueve inventario al aprobarse.
Encargos perdidos	Cada pedido especial tiene estado, abonos y saldo consultables.

7. Implementación

7.1 Estructura del proyecto

```
facturacion-app/
+-- backend/                 API REST — NestJS 11
|   +-- prisma/
|   |   +-- schema.prisma    19 modelos · 10 enumeraciones
|   |   +-- seed.ts          Datos iniciales (empresa, sedes, admin)
|   +-- src/
|       +-- auth/            JWT, guards de rol, estrategia Passport
|       +-- prisma/          Servicio de conexión (módulo global)
|       +-- products/        Productos y búsqueda por código de barras
|       +-- categories/      Categorías
|       +-- inventory/       Existencias, movimientos, ajustes y traslados
|       +-- sales/           Facturación
|       +-- customers/       Clientes
|       +-- loans/           Créditos, abonos, cambios y devoluciones
|       +-- cash/            Turnos de caja y cuadre
|       +-- branches/        Sedes
|       +-- transfers/       Solicitudes de traslado entre sedes
|       +-- special-orders/  Pedidos especiales
|       +-- purchase-list/   Lista de compras y motor de recomendación
|       +-- price-requests/  Solicitudes de cambio de precio
|       +-- reports/         Informes gerenciales
|       +-- telegram/        Vinculación y notificaciones
|
+-- frontend/                SPA / PWA — React 19 + Vite 8
|   +-- src/
|       +-- pages/           10 pantallas (una por módulo)
|       +-- components/
|       |   +-- ui/          Biblioteca de componentes reutilizables
|       |   +-- facturacion/ Formularios de venta, clientes y préstamos
|       |   +-- caja/        Apertura, cierre y resumen del turno
```

```
|       |   +-- informes/    Paneles de indicadores
|       |   +-- pedidos/     Gestión de pedidos especiales
|       +-- stores/          Estado global (sesión, sede activa)
|       +-- lib/             Cliente HTTP, formatos, tipos, PDF
|
+-- docs/                    Documentación técnica complementaria
```

7.2 Componentes implementados

Backend — 16 módulos registrados en AppModule, todos protegidos con JwtAuthGuard y, donde corresponde, RolesGuard:

Módulo	Endpoints principales
auth	POST /auth/login, POST /auth/refresh, GET /auth/me
products	CRUD, GET /products/barcode
categories	CRUD
inventory	GET /inventory, /movements, /low-stock, POST /adjust, /adjust-bulk, /transfer
sales	POST /sales, listado, detalle, POST /:id/cancel, GET /:id/payments
customers	Listado, creación y edición
loans	Listado, POST /:id/payments, POST /:id/exchange, anulación
cash	GET /cash/current, POST /open, POST /close, GET /:id/summary, /:id/discounts
transfers	POST /transfers, listado, POST /:id/approve, /:id/reject
special-orders	CRUD, abonos, cambio de estado
purchase-list	GET /recommendations, listado, creación, POST /from-recommendation, resolución
price-requests	Creación, listado, POST /:id/approve, /:id/reject
reports	sales-summary, top-products, inventory-status, payment-summary, general-summary, profit-summary, profit-total
telegram	POST /link-code, GET /status, DELETE /links/:id, POST /webhook

branches	Listado de sedes
prisma	Servicio de conexión (global)

Frontend — 10 páginas: Login, Dashboard, Caja, Inventario, Facturación, Informes, Traslados, Pedidos especiales, Lista de compras y Solicitudes de precio, más la pantalla de vinculación con Telegram.

7.3 Detalles técnicos destacados

Transaccionalidad de la venta. El registro de una venta ejecuta dentro de una sola transacción de Prisma: creación de la factura con su numeración consecutiva por sede, inserción de los detalles, registro de los pagos, descuento del inventario, generación de los movimientos de stock y —cuando es a crédito— creación del préstamo. Si cualquier paso falla, no se aplica ninguno.

Validación de coherencia de pagos. En venta de contado, el sistema exige que la suma de los pagos coincida con el total, con tolerancia de un centavo. En venta a crédito, exige que quede saldo pendiente: si el cliente paga completo, la venta debe registrarse como de contado.

Bloqueo de venta sin caja abierta. El cajero no puede facturar si no ha abierto su turno, condición que garantiza que toda venta quede vinculada a un turno auditable. El administrador queda exento de esta restricción.

Motor de recomendación de compras. Calcula, por combinación de sede y producto, las unidades vendidas en una ventana de 30 días, deriva la velocidad diaria y proyecta los días restantes hasta el agotamiento. Marca como recomendado todo producto con cinco unidades o menos, con agotamiento proyectado dentro de siete días, o con diez o más unidades vendidas en la ventana. Excluye lo que ya está en la lista pendiente y ordena por urgencia. El cálculo se hace en vivo y no persiste nada hasta que un usuario confirma la sugerencia.

Importación masiva desde Excel. La lectura del archivo se realiza en el navegador con SheetJS mediante importación dinámica —para no cargar la biblioteca en el paquete inicial—. El archivo requiere solo dos columnas (codigo_barras y cantidad); la operación y las sedes se eligen en la interfaz para reducir errores de digitación. Los códigos inexistentes se marcan y bloquean el guardado. Al confirmar, se invoca una sola vez el endpoint masivo correspondiente, que es transaccional.

Cálculo de ganancia. La ganancia se obtiene por línea vendida como la diferencia entre el subtotal cobrado y el precio de compra del producto. Los pedidos especiales aportan ganancia solo cuando ya fueron entregados y tienen costo registrado, para no inflar el resultado con costos en cero. El total agregado se resuelve en SQL, sin cargar las ventas en memoria, para que el indicador responda con rapidez sobre volúmenes grandes.

Notificaciones de caja. La apertura y el cierre disparan un mensaje a los chats de Telegram de los usuarios ADMIN y SUPERVISOR vinculados. El cierre incluye el informe completo del turno con la

diferencia expresada como sobrante, faltante o cuadre exacto. Los envíos son *best-effort*: un fallo de Telegram no afecta la operación de caja. Las horas se expresan en la zona America/Bogota.

Generación de documentos PDF. La lista de compras se exporta a PDF con jspdf y jspdf-autotable, en un formato orientado al proveedor.

7.4 Stack tecnológico

Capa	Tecnología	Versión
Lenguaje	TypeScript	5.7 (backend) / 6.0 (frontend)
Backend	NestJS · Node.js	11
ORM	Prisma + adaptador pg	7.8
Base de datos	PostgreSQL (Supabase)	—
Autenticación	@nestjs/jwt · Passport JWT · bcrypt	—
Validación	class-validator · class-transformer	—
Frontend	React · React Router	19 · 7
Construcción	Vite	8
Estilos	Tailwind CSS	4.3
Estado	Zustand	5
PWA	vite-plugin-pwa (Workbox)	1.3
Documentos	jsPDF · jspdf-autotable · SheetJS	—
Pruebas	Jest · Supertest	30 · 7
Gestor de paquetes	pnpm	—

7.5 Despliegue

Componente	Servicio
API	Render
Base de datos	Supabase (PostgreSQL)
Aplicación web	Vercel

Backend y frontend son proyectos independientes, sin package.json raíz ni enlace de espacio de trabajo, lo que permite desplegarlos y versionarlos por separado.

7.6 Puesta en marcha

```
# Backend
cd backend
pnpm install
pnpm prisma generate
pnpm prisma migrate dev
pnpm start:dev          # http://localhost:3000


# Frontend
cd frontend
pnpm install
pnpm dev                # http://localhost:5173
```

Variables de entorno requeridas en backend/.env:

Variable	Propósito
DATABASE_URL	Conexión agrupada (pgBouncer, puerto 6543)
DIRECT_URL	Conexión directa (puerto 5432) — usada por el adaptador y las migraciones
JWT_SECRET	Secreto de firma de los tokens de acceso
TELEGRAM_BOT_TOKEN	Token del bot (opcional; habilita las notificaciones)
TELEGRAM_BOT_USERNAME	Nombre del bot mostrado en la pantalla de vinculación
TELEGRAM_WEBHOOK_SECRET	Secreto de verificación del webhook (recomendado)

En el frontend, VITE_BASE_URL debe apuntar a la URL del backend.

---

8. Justificación del proyecto

8.1 Justificación operativa

El argumento central para desarrollar este sistema es que **el volumen de información que genera un negocio con dos sedes, alta rotación y venta a crédito supera la capacidad de gestión manual**. No se trata de una preferencia por la tecnología: es un umbral. Un negocio de una sola sede, catálogo corto y venta exclusivamente de contado puede administrarse con un cuaderno. Cuando se agregan una segunda sede con inventario propio, un catálogo de cientos de referencias, cartera de créditos, encargos de clientes y traslados entre puntos, el número de estados que hay que mantener sincronizados crece hasta hacer inviable el registro manual: el error deja de ser un accidente y se vuelve estadísticamente inevitable.

Cada uno de los problemas del diagnóstico tiene un costo económico directo:

- El **inventario desincronizado** genera ventas perdidas por agotados que se creían disponibles, y capital inmovilizado en referencias que se creían agotadas.
- El **descuadre de caja no explicable** produce pérdidas que no pueden atribuirse ni corregirse, y erosiona la confianza con el personal.
- La **cartera sin control** inmoviliza capital de trabajo en deudas que nadie persigue porque nadie las tiene cuantificadas.
- Las **compras por intuición** ocasionan simultáneamente sobrestock y agotados, que son las dos formas de perder dinero en inventario.
- Los **encargos perdidos** cuestan el cliente, no solo la venta.

8.2 Justificación técnica

La elección del stack responde a criterios de sostenibilidad más que de novedad:

- **Un solo lenguaje en toda la pila** reduce el costo de mantenimiento y la barrera de entrada para quien deba continuar el desarrollo.
- **TypeScript estricto y PostgreSQL con restricciones declarativas** establecen dos capas de protección independientes sobre datos financieros: el compilador y el motor de base de datos.
- **NestJS con arquitectura modular** permite que el sistema crezca por adición de módulos y no por modificación de los existentes, lo que se evidencia en que sus 16 módulos se desarrollaron de forma incremental sin reestructuraciones.
- **PWA en lugar de aplicación nativa** elimina el costo de desarrollar y mantener dos aplicaciones móviles, y la fricción de distribuirlas por tiendas, sin sacrificar la experiencia de aplicación instalada.
- **Servicios gestionados** (Supabase, Render, Vercel) evitan la administración de infraestructura, que un negocio de este tamaño no puede sostener.

8.3 Justificación de la seguridad y la trazabilidad

En un sistema que maneja dinero, inventario y cartera, la seguridad no es un requisito accesorio. El diseño responde a tres riesgos concretos:

1. **Acceso indebido.** Autenticación con tokens de vida corta, refresco rotativo y revocable, contraseñas cifradas con bcrypt, y autorización por rol verificada en el servidor —no solo ocultando opciones en la interfaz—.
2. **Manipulación de datos.** Validación estricta en la frontera de entrada, transaccionalidad en las operaciones compuestas y restricciones de integridad en el motor de base de datos.
3. **Falta de auditoría.** Cada movimiento de inventario queda tipado y fechado; cada venta se vincula a su usuario, su sede y su turno; cada aprobación registra quién la resolvió y cuándo. El sistema no solo previene errores: permite reconstruir lo ocurrido cuando se producen.

8.4 Justificación social y de escalabilidad

El sistema formaliza la operación de un negocio que compite con cadenas que ya cuentan con herramientas de este tipo. Reduce la carga administrativa del personal, elimina la fricción por descuadres no explicables, y entrega al cliente comprobantes y seguimiento verificables de sus compras, abonos y encargos.

La arquitectura está diseñada para crecer. El modelo Company → Branch es multiempresa desde el diseño y no como añadido posterior: agregar una tercera sede no requiere cambios de código, solo un registro. La separación entre API y cliente permite incorporar en el futuro una aplicación móvil nativa, la integración con facturación electrónica DIAN o un portal de consulta para clientes, sin reescribir el núcleo del sistema.

8.5 Viabilidad

Dimensión	Evaluación
Técnica	Stack maduro, ampliamente documentado y con comunidad activa. Sistema funcional y desplegado.
Económica	Sin licencias de software. Los servicios de despliegue ofrecen planes gratuitos o de bajo costo suficientes para el volumen actual.
Operativa	Interfaz en español, adaptada al flujo real de trabajo del negocio. Restricciones por rol que reducen el margen de error del usuario.
Legal	Registro completo de operaciones que sirve de base para el cumplimiento tributario y para una futura integración con facturación electrónica.

Documentación complementaria

Documento	Contenido
docs/telegram.md	Configuración del bot, vinculación de cuentas y notificaciones de caja
docs/importacion-excel.md	Diseño de la importación masiva de inventario
CLAUDE.md	Guía técnica de arquitectura y comandos
FEEDBACK.md	Revisión de arquitectura y oportunidades de mejora